[2] 3m
```
! pip install kaggle
from google.colab import drive
drive.mount('/content/drive')
! mkdir ~/.kaggle
!cp kaggle.json ~/.kaggle
!chmod 600 ~/.kaggle/kaggle.json
! kaggle datasets download 'paultimothymooney/breast-histopathology-images'
! unzip  breast-histopathology-images.zip
```

```
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1751_y951_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y1001_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y1051_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y601_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y651_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y701_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y751_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y801_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y851_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y901_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1801_y951_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y1001_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y1051_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y601_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y651_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y701_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y751_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y801_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y851_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y901_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1851_y951_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y1001_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y1051_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y601_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y651_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y701_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y751_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y801_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y851_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y901_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1901_y951_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y1001_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y1051_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y601_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y651_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y701_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y751_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y801_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y851_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y901_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x1951_y951_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y1001_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y1051_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y701_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y751_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y801_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y851_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y901_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2001_y951_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y1001_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y1051_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y801_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y851_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y901_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2051_y951_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2101_y1001_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2101_y901_class1.png
  inflating: IDC_regular_ps50_idx5/9383/1/9383_idx5_x2101_y951_class1.png
```

[3] 8s
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import cv2
import glob
import random
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

[4] 10s
```
!pip install keras_efficientnets
```

```
Collecting keras_efficientnets
  Downloading keras_efficientnets-0.1.7-py2.py3-none-any.whl (15 kB)
Requirement already satisfied: keras>=2.2.4 in /usr/local/lib/python3.10/dist-packages (from keras_efficientnets) (2.15.0)
Requirement already satisfied: scipy>=1.1.0 in /usr/local/lib/python3.10/dist-packages (from keras_efficientnets) (1.11.4)
Requirement already satisfied: scikit-learn>=0.21.2 in /usr/local/lib/python3.10/dist-packages (from keras_efficientnets) (1.2.2)
Requirement already satisfied: numpy>=1.17.3 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.21.2->keras_efficientnets) (1.23.5)
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.21.2->keras_efficientnets) (1.3.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.21.2->keras_efficientnets) (3.2.0)
Installing collected packages: keras_efficientnets
Successfully installed keras_efficientnets-0.1.7
```

[6] 7s
```
!pip install keras_applications
```

```
Collecting keras_applications
  Downloading Keras_Applications-1.0.8-py3-none-any.whl (50 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 50.7/50.7 kB 1.3 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.9.1 in /usr/local/lib/python3.10/dist-packages (from keras_applications) (1.23.5)
Requirement already satisfied: h5py in /usr/local/lib/python3.10/dist-packages (from keras_applications) (3.9.0)
Installing collected packages: keras_applications
Successfully installed keras_applications-1.0.8
```

```
[8] !pip install efficientnet
```

```
Collecting efficientnet
  Downloading efficientnet-1.1.1-py3-none-any.whl (18 kB)
Requirement already satisfied: keras-applications<=1.0.8,>=1.0.7 in /usr/local/lib/python3.10/dist-packages (from efficientnet) (1.0.8)
Requirement already satisfied: scikit-image in /usr/local/lib/python3.10/dist-packages (from efficientnet) (0.19.3)
Requirement already satisfied: numpy>=1.9.1 in /usr/local/lib/python3.10/dist-packages (from keras-applications<=1.0.8,>=1.0.7->efficientnet) (1.23.5)
Requirement already satisfied: h5py in /usr/local/lib/python3.10/dist-packages (from keras-applications<=1.0.8,>=1.0.7->efficientnet) (3.9.0)
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (1.11.4)
Requirement already satisfied: networkx>=2.2 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (3.2.1)
Requirement already satisfied: pillow!=7.1.0,!=7.1.1,!=8.3.0,>=6.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (9.4.0)
Requirement already satisfied: imageio>=2.4.1 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (2.31.6)
Requirement already satisfied: tifffile>=2019.7.26 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (2023.12.9)
Requirement already satisfied: PyWavelets>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from scikit-image->efficientnet) (23.2)
Installing collected packages: efficientnet
Successfully installed efficientnet-1.1.1
```

```
[ ] Start coding or generate with AI.
```

```
[10] from efficientnet.tfkeras import EfficientNetB0  # or choose the desired EfficientNet variant
     from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, BatchNormalization, Dropout
     from tensorflow.keras.models import Model
```

```
[11] from google.colab import drive
     drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
[12] breast_img = glob.glob('/content/IDC_regular_ps50_idx5/**/*.png', recursive = True)

     for imgname in breast_img[:3]:
         print(imgname)
```

```
/content/IDC_regular_ps50_idx5/9174/1/9174_idx5_x1551_y901_class1.png
/content/IDC_regular_ps50_idx5/9174/1/9174_idx5_x1401_y1101_class1.png
/content/IDC_regular_ps50_idx5/9174/1/9174_idx5_x1501_y801_class1.png
```

```
[13] # Split images based on class (IDC negative and IDC positive)
     N_IDC = [img for img in breast_img if img[-5] == '0']
     P_IDC = [img for img in breast_img if img[-5] == '1']
```

```
[14] # Reduce the number of IDC (-) samples to balance the classes
     NewN_IDC = N_IDC[:78786]
```

```
[15] # Data Preprocessing
     non_img_arr = []
     can_img_arr = []
```

```
[16] # Process IDC (-) images
     for img in NewN_IDC:
         n_img = cv2.imread(img, cv2.IMREAD_COLOR)
         n_img_size = cv2.resize(n_img, (50, 50), interpolation=cv2.INTER_LINEAR)
         non_img_arr.append([n_img_size, 0])
```

```
[17] # Process IDC (+) images
     for img in P_IDC:
         c_img = cv2.imread(img, cv2.IMREAD_COLOR)
         c_img_size = cv2.resize(c_img, (50, 50), interpolation=cv2.INTER_LINEAR)
         can_img_arr.append([c_img_size, 1])
```

```
[18] # Combine the processed images and shuffle
     breast_img_arr = np.concatenate((non_img_arr[:12389], can_img_arr[:12389]))
     random.shuffle(breast_img_arr)

     X = np.array([feature for feature, _ in breast_img_arr])
     y = np.array([label for _, label in breast_img_arr])
```

```
[19] # Train-test split
     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

```
[20] # Normalize pixel values
     X_train = X_train / 255.0
     X_test = X_test / 255.0
```

```
[21] efficientnet_base = EfficientNetB0(weights='imagenet', include_top=False, input_shape=(50, 50, 3))
```

```
Downloading data from https://github.com/Callidior/keras-applications/releases/download/efficientnet/efficientnet-b0_weights_tf_dim_ordering_tf_kernels_autoaugment_notop.h5
16804768/16804768 [==============================] - 0s 0us/step
```

```
[22] for layer in efficientnet_base.layers:
         layer.trainable = False
```

```
[23] x = efficientnet_base.output
     x = GlobalAveragePooling2D()(x)
```

```
x = Dense(128, activation='relu', kernel_initializer='he_uniform')(x)
x = BatchNormalization()(x)
x = Dropout(0.5)(x)
predictions_efficientnet = Dense(1, activation='sigmoid')(x)

model_efficientnet = Model(inputs=efficientnet_base.input, outputs=predictions_efficientnet)
```

[25]
```
from tensorflow.keras.optimizers import Adam

model_efficientnet.compile(optimizer=Adam(learning_rate=0.0001), loss='binary_crossentropy', metrics=['accuracy'])
```

[27]
```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    vertical_flip=True,
    fill_mode='nearest'
)

train_generator = train_datagen.flow(X_train, y_train, batch_size=35)
```

[28]
```
history_efficientnet = model_efficientnet.fit(train_generator, epochs=200, validation_data=(X_test, y_test))
```

```
Epoch 172/200
496/496 [==============================] - 25s 50ms/step - loss: 0.2352 - accuracy: 0.9061 - val_loss: 0.2413 - val_accuracy: 0.9061
Epoch 173/200
496/496 [==============================] - 23s 47ms/step - loss: 0.2343 - accuracy: 0.9068 - val_loss: 0.2431 - val_accuracy: 0.9056
Epoch 174/200
496/496 [==============================] - 26s 52ms/step - loss: 0.2319 - accuracy: 0.9085 - val_loss: 0.2758 - val_accuracy: 0.8882
Epoch 175/200
496/496 [==============================] - 23s 47ms/step - loss: 0.2379 - accuracy: 0.9046 - val_loss: 0.2453 - val_accuracy: 0.9030
Epoch 176/200
496/496 [==============================] - 25s 51ms/step - loss: 0.2364 - accuracy: 0.9061 - val_loss: 0.2503 - val_accuracy: 0.8986
Epoch 177/200
496/496 [==============================] - 24s 49ms/step - loss: 0.2381 - accuracy: 0.9067 - val_loss: 0.2446 - val_accuracy: 0.9044
Epoch 178/200
496/496 [==============================] - 24s 48ms/step - loss: 0.2376 - accuracy: 0.9043 - val_loss: 0.2347 - val_accuracy: 0.9092
Epoch 179/200
496/496 [==============================] - 25s 51ms/step - loss: 0.2347 - accuracy: 0.9090 - val_loss: 0.2472 - val_accuracy: 0.9013
Epoch 180/200
496/496 [==============================] - 25s 50ms/step - loss: 0.2368 - accuracy: 0.9067 - val_loss: 0.2320 - val_accuracy: 0.9080
Epoch 181/200
496/496 [==============================] - 24s 47ms/step - loss: 0.2321 - accuracy: 0.9087 - val_loss: 0.2292 - val_accuracy: 0.9109
Epoch 182/200
496/496 [==============================] - 25s 51ms/step - loss: 0.2342 - accuracy: 0.9076 - val_loss: 0.2371 - val_accuracy: 0.9068
Epoch 183/200
496/496 [==============================] - 23s 47ms/step - loss: 0.2401 - accuracy: 0.9064 - val_loss: 0.2370 - val_accuracy: 0.9072
Epoch 184/200
496/496 [==============================] - 26s 52ms/step - loss: 0.2296 - accuracy: 0.9105 - val_loss: 0.2410 - val_accuracy: 0.9061
Epoch 185/200
496/496 [==============================] - 25s 50ms/step - loss: 0.2336 - accuracy: 0.9079 - val_loss: 0.2395 - val_accuracy: 0.9060
Epoch 186/200
496/496 [==============================] - 24s 49ms/step - loss: 0.2357 - accuracy: 0.9059 - val_loss: 0.2348 - val_accuracy: 0.9101
Epoch 187/200
496/496 [==============================] - 25s 51ms/step - loss: 0.2350 - accuracy: 0.9071 - val_loss: 0.2539 - val_accuracy: 0.8976
Epoch 188/200
496/496 [==============================] - 26s 53ms/step - loss: 0.2391 - accuracy: 0.9055 - val_loss: 0.2363 - val_accuracy: 0.9076
Epoch 189/200
496/496 [==============================] - 24s 47ms/step - loss: 0.2378 - accuracy: 0.9049 - val_loss: 0.2502 - val_accuracy: 0.9003
Epoch 190/200
496/496 [==============================] - 26s 52ms/step - loss: 0.2344 - accuracy: 0.9065 - val_loss: 0.2429 - val_accuracy: 0.9041
Epoch 191/200
496/496 [==============================] - 24s 48ms/step - loss: 0.2312 - accuracy: 0.9069 - val_loss: 0.2482 - val_accuracy: 0.8987
Epoch 192/200
496/496 [==============================] - 24s 49ms/step - loss: 0.2342 - accuracy: 0.9058 - val_loss: 0.2418 - val_accuracy: 0.9048
Epoch 193/200
496/496 [==============================] - 27s 53ms/step - loss: 0.2386 - accuracy: 0.9054 - val_loss: 0.2333 - val_accuracy: 0.9107
Epoch 194/200
496/496 [==============================] - 28s 56ms/step - loss: 0.2361 - accuracy: 0.9072 - val_loss: 0.2548 - val_accuracy: 0.8947
Epoch 195/200
496/496 [==============================] - 24s 48ms/step - loss: 0.2317 - accuracy: 0.9088 - val_loss: 0.2516 - val_accuracy: 0.8984
Epoch 196/200
496/496 [==============================] - 30s 59ms/step - loss: 0.2345 - accuracy: 0.9057 - val_loss: 0.2381 - val_accuracy: 0.9065
Epoch 197/200
496/496 [==============================] - 26s 53ms/step - loss: 0.2343 - accuracy: 0.9059 - val_loss: 0.2339 - val_accuracy: 0.9111
Epoch 198/200
496/496 [==============================] - 24s 49ms/step - loss: 0.2331 - accuracy: 0.9072 - val_loss: 0.2449 - val_accuracy: 0.9019
Epoch 199/200
496/496 [==============================] - 24s 48ms/step - loss: 0.2335 - accuracy: 0.9064 - val_loss: 0.2492 - val_accuracy: 0.8979
Epoch 200/200
496/496 [==============================] - 27s 53ms/step - loss: 0.2327 - accuracy: 0.9068 - val_loss: 0.2470 - val_accuracy: 0.8986
```

[30]
```
model_efficientnet.evaluate(X_test, y_test)
```

```
233/233 [==============================] - 3s 12ms/step - loss: 0.2470 - accuracy: 0.8986
[0.24695809185504913, 0.8985741138458252]
```

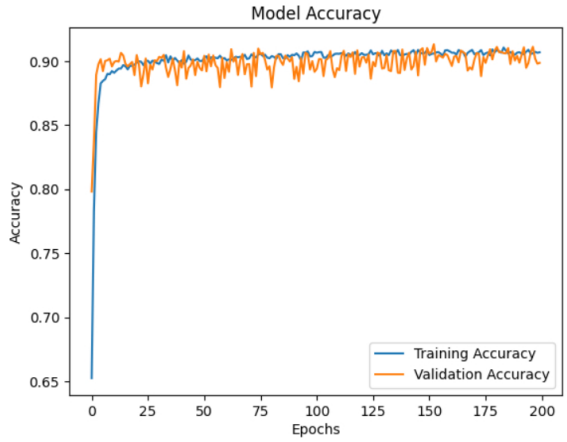
[31]
```
model_efficientnet.save("Brest_idc_eff_Model.h5")
```

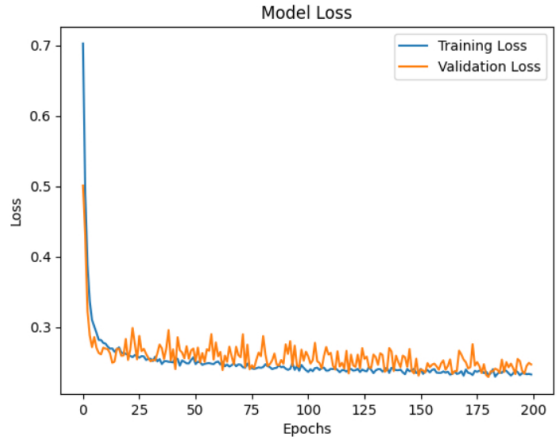
```
/usr/local/lib/python3.10/dist-packages/keras/src/engine/training.py:3103: UserWarning: You are saving your model as an HDF5 file via `model.save()`. This file format is considered lega
  saving_api.save_model(
```

[33]
```
# Plotting Accuracy and Loss
plt.plot(history_efficientnet.history['accuracy'], label='Training Accuracy')
plt.plot(history_efficientnet.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
```

```
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

plt.plot(history_efficientnet.history['loss'], label='Training Loss')
plt.plot(history_efficientnet.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```





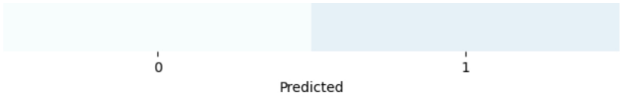
[34]
```
# Predict probabilities for the test set
y_pred_prob = model_efficientnet.predict(X_test)

# Convert probabilities to classes
y_pred = (y_pred_prob > 0.5).astype('int32')

# Create confusion matrix
cm = confusion_matrix(y_test, y_pred)
```

```
233/233 [==============================] - 5s 12ms/step
```

[35]
```
# Plot Confusion Matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='BuPu', cbar=False)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```



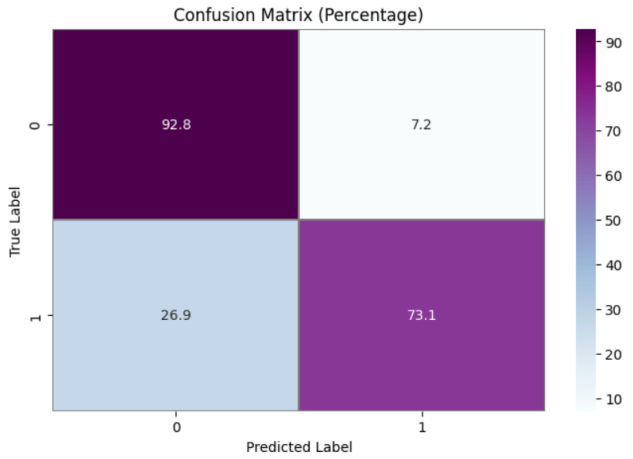
0s

```
from sklearn.metrics import accuracy_score, confusion_matrix


confusion_mtx = confusion_matrix(y_test, y_pred)

# calculate the percentage
confusion_mtx_percent = confusion_mtx.astype('float') / confusion_mtx.sum(axis=1)[:, np.newaxis] * 100

f, ax = plt.subplots(figsize=(8, 5))
sns.heatmap(confusion_mtx_percent, annot=True, linewidths=0.01, cmap="BuPu", linecolor="gray", fmt='.1f', ax=ax)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix (Percentage)")
plt.show()
```



0s completed at 5:37 PM